

A Large-Scale Analysis of AI-Assisted Pull Requests: Observations, Biases, and the Limits of Behavioral Inference

Abstract—This paper examines the empirical foundations of large-scale observational studies of AI-assisted software development. Through a multi-method analysis of pull request datasets labeled with AI agent information, we trace the trajectory from initial statistical signals to validated findings and structural diagnosis. The study identifies specific dataset properties that constrain behavioral inference, including extreme contribution inequality, automation dominance, attribution ambiguity, non-independence, and significant data loss. We demonstrate that manual validation reveals substantial heterogeneity in testing practices that is more strongly associated with user and repository context than with agent identity. Automated metadata-driven proxies for testing behavior systematically misclassify test presence when compared against manual ground truth. While these limitations preclude defensible agent-level behavioral claims, the analysis shows that certain structural and distributional properties remain empirically interpretable. Based on these findings, we provide methodological guidance for future research, emphasizing validation-first study design, explicit measurement error quantification, and dataset construction practices necessary for reliable empirical analysis of AI-assisted development.

I. INTRODUCTION

AI-powered coding assistants are now embedded in everyday software development workflows. Tools such as GitHub Copilot, OpenAI Codex, Cursor, Devin, and Claude Code are used by millions of developers and are increasingly credited with accelerating development, improving productivity, and influencing software quality practices. As these tools become pervasive, understanding how they shape real-world development behavior—particularly testing practices—has become an important empirical question for both researchers and practitioners.

Recent software engineering research has responded to this need by leveraging large-scale datasets derived from development platforms such as GitHub. These datasets appear especially attractive: they capture real production activity at unprecedented scale, promise high statistical power, and seemingly allow direct comparison of AI agent behavior “in the wild”. In particular, pull request (PR) datasets labeled with AI agent information have been used to make claims about differences in testing behavior, development style, and quality practices across tools.

However, as our analysis demonstrates, initial automated detection suggests high overall test rates (93.49% across 932,791 PRs) with apparent agent differences ranging from 98.5% for OpenAI Codex to 18.8% for Cursor. These statistical patterns initially appear compelling but dissolve under manual

validation, which reveals systematic measurement error and structural dataset biases that undermine behavioral inference.

This study began with exactly that motivation. Our original goal was to determine whether different AI coding agents exhibit systematically different testing behaviors. Using a large PR dataset with agent labels, we initially observed striking patterns: some agents appeared to include tests in nearly all PRs, while others rarely did so. Standard statistical analysis suggested large, statistically significant differences, seemingly supporting the narrative that AI agents differ meaningfully in how they support testing.

To conduct this initial analysis, we made several key assumptions: (1) pull requests are independent observations, (2) agent labels accurately reflect the AI tool used without human modification, (3) keyword-based detection of testing (e.g., presence of “test” in PR titles or descriptions) reliably indicates actual test inclusion, and (4) the dataset represents a random sample of AI-assisted development activity. Under these assumptions, we performed chi-square tests comparing test presence rates across agents. The results showed highly significant differences ($p < 0.001$), with test inclusion rates ranging from approximately 15% for some agents to over 85% for others, corresponding to large effect sizes under conventional benchmarks for categorical data.

However, as the analysis progressed, these early findings raised a more fundamental question: what exactly do these apparent differences represent?

Closer inspection of the data revealed unexpected and troubling characteristics. Contribution activity was extremely concentrated, with a small number of users responsible for a disproportionate fraction of PRs. Some agent categories were dominated by single accounts producing thousands of PRs. A substantial portion of PRs were no longer accessible, preventing direct verification of their contents. Manual inspection further showed that keyword-based indicators of testing often failed to correspond to actual test files or meaningful test quality.

These observations forced a reassessment of the original research question. Rather than asking whether AI agents behave differently, this work reframes the problem: can such PR datasets, as currently constructed, support valid agent behavior analysis at all?

In response, this paper presents a structured empirical investigation that combines large-scale quantitative analysis with targeted manual validation. We document what the dataset

clearly shows—such as adoption patterns and extreme contribution inequality—while also identifying where inference breaks down due to attribution ambiguity, measurement error, automation, and missing data. Rather than advancing unsupported behavioral claims, this study offers a methodological account of how seemingly compelling results emerge, why they are fragile, and how future research can avoid similar pitfalls.

By telling the full analytical story—from initial hypothesis, through failure, to methodological insight—this work contributes both empirical observations and a cautionary framework for AI tool research based on large observational datasets.

A. Research Questions

This study investigates whether large-scale observational pull request datasets labeled with AI agent information can support valid agent behavior claims. Specifically, it examines (i) whether such datasets satisfy the assumptions required for agent-level behavioral comparison, (ii) which structural and measurement properties undermine inference, (iii) the reliability of metadata- and keyword-based proxies for testing behavior, and (iv) which empirical properties remain valid despite these limitations. Based on these findings, the study derives methodological guidance for future empirical research on AI-assisted software development.

To assess testing habits as an indicator of code quality, we address the following research questions based on manual validation: (i) How frequently do coding agents contribute tests? What types (e.g., unit, integration, end-to-end) are most common? (ii) What is the test-to-code churn ratio across ecosystems? (iii) When tests are missing in initial agentic-PRs, do developers intervene to ensure reliable software testing (via follow-up commits or related PRs)?

II. DATASET ANALYSIS

A. Dataset Source

This study uses the AIDev dataset released as part of the MSR 2026 Mining Challenge. The dataset was collected by Li et al. and made publicly available to support research on AI-assisted software development. It aggregates GitHub pull request data annotated with AI agent attribution labels, representing one of the largest publicly available datasets of AI-assisted development activity.

The dataset was obtained directly from the official MSR 2026 challenge release and was used without modification to its original labels or structure, except where explicitly noted for analysis and validation purposes.

B. Dataset Scope and Composition

The full dataset contains 932,791 pull requests, 72,189 unique users, and five AI agent labels (OpenAI Codex, GitHub Copilot, Cursor, Devin, and Claude Code). Each pull request record includes an agent label, user identifier, repository information, pull request title and description, timestamps (including creation time, closure time, and merge time where

applicable), merge status, and links to the original GitHub PR (when available).

The dataset spans a large number of repositories, programming languages, and development contexts, providing broad coverage of real-world AI-assisted development activity.

C. Intended Use and Initial Assumptions

The dataset is explicitly designed to support observational analysis of AI-assisted development practices. At face value, its scale and breadth appear well suited for behavioral comparison across agents, including analysis of testing practices via PR metadata.

However, as this study demonstrates, the suitability of the dataset for such inference depends critically on assumptions about attribution accuracy, data completeness, user independence, and measurement validity. These assumptions are examined empirically rather than taken for granted.

III. METHODOLOGY

To systematically investigate the dataset’s properties, we employed a multi-stage analytical approach combining quantitative aggregation, statistical testing, and manual validation. First, we computed distributional statistics across the full dataset, including Gini coefficients for contribution inequality and frequency distributions of PRs per user. Second, we performed accessibility checks by attempting to retrieve PR content via GitHub API and direct links, identifying 238 inaccessible PRs out of 385 sampled (62% success rate). Third, for manual validation, we stratified a sample of 385 PRs (77 per agent) to ensure representation, then applied a four-level rubric for test presence and quality: (1) No tests, (2) Minimal tests (e.g., basic assertions), (3) Moderate tests (comprehensive coverage), (4) Extensive tests (integration and edge cases). Each PR was inspected by examining the PR description, file changes, and linked commits. Manual validation was treated as the primary empirical evidence in this study. Automated detection methods were applied only after manual inspection, and solely to quantify measurement error rather than to support behavioral inference. Finally, we conducted bidirectional validation, comparing automated detection against manual ground truth to quantify measurement error rates. We did not operationalize PR lifespan as an automation proxy, as preliminary inspection showed substantial overlap between human-reviewed and automated PR durations.

A. Prototyping Hypothesis and Dataset Biases

During manual inspection, we identified systematic biases suggesting that some AI agents are predominantly used for rapid prototyping workflows. High PR volumes from few users, combined with elevated 404 error rates, indicate that many PRs originate from transient repositories created for experimentation and subsequently deleted. This “prototyping hypothesis” explains the combination of apparent agent activity with poor data persistence and questions the representativeness of aggregate statistics for production development practices.

TABLE I
DATASET SUMMARY

Metric	Value
Total PRs	932,791
Unique users	72,189
Gini coefficient	0.83
Inaccessible PRs in manual review	238
Agents	5

This hypothesis is treated as an empirical observation rather than a definitive claim, but it motivates careful interpretation of adoption metrics and behavioral patterns derived from observational PR datasets.

B. Automated Test-Detection Heuristics

To quantify measurement error, we defined explicit heuristics for automated test detection applied to PR metadata. These heuristics were developed based on common testing conventions and validated against manual inspection.

H1 (Keyword in PR title/body): Matches case-insensitive patterns including "test", "tests", "testing", "pytest", "junit", "jest", "spec", "e2e", "integration", "unit".

H2 (Changed-file path heuristic): Any changed file path contains "/test/", "/tests/", `tests/`, `/spec/`, `spec/`, `test/`.

A positive detection required H1 OR H2.

Key assumptions underlying these heuristics: A1: PR text reflects actual code changes. A2: Test-related file naming conventions correspond to tests. A3: Mentions of "tests" are not purely template boilerplate.

The dataset includes PR creation, closure, and merge timestamps, but these do not provide ground truth about authorship, human review, or whether tests were executed; therefore timestamps are used descriptively (e.g., PR lifespan distributions) rather than as behavioral evidence.

IV. RESULTS

This section reports empirical findings addressing the research questions defined in Section I-A. We lead with manually validated findings from our stratified sample, then contrast these with automated detection results to quantify measurement error. This validation-first approach ensures that behavioral observations are grounded in verified evidence rather than metadata proxies.

A. Manual Validation of Test Presence and Quality (RQ1)

A stratified manual review of 385 pull requests (77 per agent) established ground truth for test presence and quality

using a four-level rubric: (1) No tests present, (2) Minimal testing (basic unit tests or assertions), (3) Moderate testing (comprehensive unit tests with some integration), (4) Extensive testing (full test suites including integration, performance, and edge cases). From the accessible PRs (147 out of 385), we observed that 34% had no tests, 28% had minimal testing, 25% had moderate testing, and 13% had extensive testing. Testing practices varied widely, with some PRs containing well-structured test files (e.g., JUnit tests in Java repositories) while others had only informal assertions or no tests at all.

Comparison between manual inspection and keyword-based detection revealed systematic discrepancies. In 22% of cases, keyword indicators produced false positives, typically arising from boilerplate PR templates or issue descriptions referencing testing without accompanying test files. Conversely, 18% of PRs containing tests were not identified by keyword-based methods, particularly when tests were added without explicit mention in PR metadata.

B. Observed Testing Practices in Manually Validated PRs

Manual inspection revealed substantial heterogeneity in testing practices across AI-assisted pull requests, independent of agent label. While 66% of manually accessible PRs contained some form of testing, the majority of these tests were limited in scope. Minimal testing|such as basic assertions or single test cases|accounted for 28% of PRs, whereas only 13% included extensive test suites covering integration scenarios or edge cases.

Testing quality varied widely even within the same agent category and repository. In several cases, PRs attributed to different agents within the same project exhibited nearly identical testing structures, suggesting that repository conventions and user practices exert a stronger influence on testing behavior than agent identity. Conversely, PRs attributed to the same agent but produced by different users showed markedly different testing depth, ranging from no tests to comprehensive suites.

TABLE II
MANUAL VS KEYWORD-BASED TEST DETECTION

Metric	Value
Manual test presence (%)	66
Keyword-detected test presence (%)	80
False positive rate (%)	22
False negative rate (%)	18

These observations indicate that testing behavior in AI-assisted pull requests is highly context-dependent and user-driven. While AI tools may facilitate test generation, the presence and quality of tests appear to be shaped primarily by developer practices and project norms rather than by the specific coding agent used.

C. Temporal Characteristics of Manually Inspected PRs

Analysis of pull request creation and closure timestamps within the manually validated sample showed substantial overlap between PRs exhibiting strong automation signals and those with evidence of human review. While some PRs were merged within minutes of creation, others remained open for extended periods regardless of agent attribution. No consistent temporal pattern emerged that reliably distinguished automated from human-assisted PRs. This suggests that PR lifespan alone is an unreliable proxy for automation intensity in observational datasets.

D. Error Analysis: Why Heuristics Fail

To understand measurement error, we categorized false positives and negatives from the manual sample.

False positives (22% of keyword-detected PRs): - Boilerplate PR templates mentioning "Tests: [] added" without code changes. - Issue descriptions referencing testing in context (e.g., "This fixes the test failure") but no test files added. - Documentation changes mistaken for test additions.

False negatives (18% of manually validated test PRs): - Tests added without mention in PR metadata (e.g., silent file additions). - Non-standard test naming (e.g., custom frameworks not matching heuristics). - Tests integrated into existing files without new test-specific paths.

These categories show that assumptions A1-A3 are frequently violated, with templates and implicit changes causing most errors.

E. Manual-Sample Answers to Testing Behavior Questions

Using the 147 accessible PRs, we address the motivation questions:

- (i) Frequency and types: 66% of PRs included tests, with minimal testing (unit/assertions) most common (42% of test PRs), followed by moderate (38%) and extensive (20%). Unit tests were predominant, with integration tests rare.
- (ii) Test-to-code churn: In test-containing PRs, 45% changed only test files, 35% changed both test and code, 20% changed code only. This suggests moderate test-to-code coupling.
- (iii) Developer intervention: In PRs without initial tests (34% of accessible), we observed follow-up commits adding tests in 12% of cases within the PR timeline, indicating limited but present intervention.

F. Dataset-Level Structural Characteristics (RQ1, RQ2)

Analysis of the full dataset (932,791 pull requests) reveals extreme structural imbalance in contribution activity. Contribution inequality is severe, with the top 1% of contributors responsible for over 40% of all PRs. The computed Gini coefficient (≈ 0.83) indicates an exceptionally concentrated distribution. This concentration is not uniform across agents, with certain categories dominated by single users producing thousands of PRs.

G. Accessibility and Data Loss (RQ2)

A manual review of 385 pull requests revealed that 238 were inaccessible (404 errors), especially prevalent among high-volume users. This loss limits verification and distorts sampling, as accessibility is not uniformly distributed.

H. Automated Test Detection Results

To establish baseline detection rates across the full dataset, we applied keyword-based and convention-based heuristics to all 932,791 PRs. These automated methods revealed substantial apparent heterogeneity in testing behavior

across agents, with overall test detection rates of 93.49%.

TABLE III
AUTOMATED TEST DETECTION RATES BY AGENT LABEL. THESE RATES REFLECT HEURISTIC DETECTION OF TEST-RELATED ARTIFACTS AND DO NOT IMPLY VERIFIED TEST QUALITY.

Agent	Test Rate (%)	PR Count
OpenAI Codex	98.5	814,522
Copilot	79.7	50,447
Claude Code	76.8	5,137
Devin	66.6	29,744
Cursor	18.8	32,941

These results initially suggested strong distributional differences in keyword-detected test presence across agent labels. However, statistical tests comparing these rates (chi-square = 389,068.7, $p < 0.001$, Cramér's $V = 0.646$) must be interpreted cautiously due to severe violations of independence assumptions. The dataset exhibits extreme user concentration, with Devin PRs generated entirely by a single user and Copilot PRs dominated by 379 users, invalidating standard statistical inference.

I. Measurement Error in Automated Detection

Bidirectional validation shows that automated test detection systematically overestimates test presence, varying by language and repository conventions. This reflects inherent limitations in PR-level metadata.

J. Debiasing Effects on Aggregate Patterns

Applying user-level filtering alters aggregate adoption statistics, shifting agent shares and increasing distributional diversity. Raw PR counts are unstable indicators of adoption or behavior.

K. Summary of Empirical Findings

The empirical analysis yields several concrete observations about AI-assisted pull requests. Contribution activity is highly concentrated, testing practices are heterogeneous and often minimal, and testing quality varies more strongly by user and repository context than by agent label. Automated metadata-based detection methods systematically misclassify test presence when compared against manual inspection. These results characterize dataset-level and user-level properties

that constrain agent-level inference. For example, initial chi-square tests comparing keyword-detected test presence across agents yielded statistically significant differences ($p < 0.001$), but these results were invalidated after accounting for user-level dependence and measurement error.

L. Dataset Structural Properties and User Adoption Patterns (RQ2)

The dataset exhibits extreme structural biases that constrain behavioral inference about AI agents. User adoption patterns show dramatic inequality, with OpenAI Codex adopted by 61,653 unique users while Devin was used exclusively by a single user. Copilot shows moderate concentration (379 users), while Cursor and Claude Code demonstrate broader adoption (9,658 and 1,643 users respectively).

TABLE IV
USER ADOPTION PATTERNS BY AGENT LABEL, SHOWING CONCENTRATION RATIOS (PRs PER USER). EXTREME CONCENTRATION SUGGESTS SPECIALIZED USAGE MODELS RATHER THAN BROAD ADOPTION.

Agent	Unique Users	Concentration Ratio
OpenAI Codex	61,653	13.2 PRs/user
Copilot	379	133.1 PRs/user
Claude Code	1,643	3.1 PRs/user
Devin	1	29,744 PRs/user
Cursor	9,658	3.4 PRs/user

These patterns suggest differential usage models: OpenAI Codex and Cursor appear integrated into broad development workflows, while Devin and Copilot show patterns consistent with specialized or high-volume usage. The single-user dominance for Devin raises concerns about data provenance, representativeness, and the extent to which these PRs reflect general usage rather than a controlled or experimental deployment.

Repository accessibility further compounds these issues, with 62% of manually sampled PRs returning 404 errors. Inaccessible repositories were disproportionately associated with high-volume, low-user-count agents, supporting the hypothesis that some AI agents are predominantly used for rapid prototyping rather than sustained development.

V. THREATS TO VALIDITY

This section outlines threats to validity following standard empirical software engineering practice.

A. Internal Validity

Internal validity is threatened by attribution ambiguity, as agent labels indicate association, not authorship. Human modification and automation contamination complicate interpretation. Manual verification involves subjective judgment.

B. Construct Validity

Construct validity is limited by operationalization of testing behavior, focusing on presence and quality rather than effectiveness. Keyword-based detection exhibits false positives and negatives, mitigated but not eliminated by manual validation.

C. External Validity

External validity is constrained by the dataset's GitHub scope and temporal snapshot. AI tools evolve rapidly, and inaccessible PRs introduce uncertainty about representativeness. Additionally, findings from the manual sample of 385 PRs may not generalize to the entire dataset of 932,791 PRs due to sampling bias toward accessible PRs and potential differences in user behavior across agents.

D. Conclusion Validity

Conclusion validity is threatened by structural dependence, violating independence assumptions. Descriptive statistics remain valid, but inferential statistics cannot support behavioral differences.

VI. IMPACT AND IMPLICATIONS

This work impacts empirical research practice, dataset construction, and industrial interpretation of AI tool studies. Its contribution lies in establishing what can and cannot be responsibly inferred from large PR datasets.

A. Impact on Empirical Software Engineering Research

The primary impact is methodological. It demonstrates how apparently strong results can emerge from observational datasets even when core assumptions are violated. By documenting the analytical trajectory, this work provides a cautionary framework for interpreting future AI-assisted programming studies.

B. Impact on Dataset Design and Curation

The analysis reveals structural properties affecting inference, including contribution inequality, automation contamination, and accessibility loss. This provides guidance for future dataset construction, emphasizing persistent identifiers and archival preservation.

C. Impact on Industrial and Practitioner Interpretation

From an industry perspective, this work cautions against drawing operational conclusions from aggregate PR statistics. Apparent differences may reflect dataset artifacts rather than intrinsic tool characteristics.

D. Broader Research Implications

This study argues for treating methodological failure as a valid research outcome, contributing to a more transparent empirical research culture.

VII. RELATED WORK AND METHODOLOGICAL FOUNDATIONS

This work draws on empirical software engineering methodology, large-scale repository datasets, and AI-assisted programming studies.

A. Empirical Software Engineering Methodology and Validity

Foundational work emphasizes valid measurement and sampling [1], [2]. Sampling practices follow precedents like Naghashzadeh et al. [3].

B. Large-Scale Observational Datasets and Repository Mining

Repository mining research highlights metadata limitations and incompleteness [4], [5], motivating accessibility checks and manual verification.

C. AI-Assisted Programming and Coding Agents

Recent studies examine AI tools' role in development [6], [7], [8], but rely on controlled experiments rather than large observational data.

D. Measurement, Testing Practices, and Statistical Foundations

Testing practices are studied via textual proxies [9], [10]. Statistical guidance emphasizes effect size over significance [11], [12].

E. Positioning of the Present Work

This study contributes methodological analysis of AI-assisted PR datasets, clarifying valid inference limits.

VIII. CONCLUSION

This paper demonstrates both what can be empirically observed in large-scale AI-assisted pull request datasets and where behavioral inference breaks down due to structural and measurement limitations.

Initial analysis revealed apparent differences in testing practices across tools, but deeper investigation showed these patterns do not support valid behavioral conclusions due to attribution ambiguity, structural dependence, and measurement limitations.

The methodological contribution lies in establishing baseline validation techniques for AI-assisted development datasets, emphasizing empirical verification over assumption-driven analysis. Future work should prioritize instrumented and longitudinal studies, repository archiving, and controlled task-based evaluations to complement observational datasets and enable defensible analysis of AI agent behavior.

REFERENCES

- [1] C. Wohlin, P. Runeson, M. H"ost, M. Ohlsson, B. Regnell, and A. Wesslén, *Experimentation in Software Engineering*. Springer, 2012.
- [2] B. Kitchenham, S. L. Pfleeger, L. Pickard, P. W. Jones, D. C. Hoaglin, K. E. Emam, and J. Rosenberg, "Preliminary guidelines for empirical research in software engineering," *IEEE Transactions on Software Engineering*, vol. 28, no. 8, pp. 721--734, 2002.
- [3] A. Naghashzadeh et al., "Developer behavior in q&a ecosystems: A large-scale empirical study," in *Proceedings of the 44th International Conference on Software Engineering*. ACM, 2022.
- [4] S. Baltes, L. Dumani, C. Treude, and S. Diehl, "Sotorrent: Reconstructing and analyzing the evolution of stack overflow posts," in *Proceedings of the 15th International Conference on Mining Software Repositories*. ACM, 2018, pp. 319--330.
- [5] M. Asaduzzaman, A. Mashiyat, C. K. Roy, and K. A. Schneider, "Answering questions about unanswered questions of stack overflow," in *Proceedings of the 10th Working Conference on Mining Software Repositories*. IEEE, 2013, pp. 97--100.
- [6] P. Vaithilingam et al., "Expectation vs. experience: Evaluating the usability of code generation tools powered by large language models," in *Proceedings of the 44th International Conference on Software Engineering*. ACM, 2022.
- [7] J. Finnie-Ansley et al., "Robots for the masses: A survey of code generation tools powered by large language models," in *Proceedings of the 44th International Conference on Software Engineering*. ACM, 2022.
- [8] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. de Oliveira Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman et al., "Evaluating large language models trained on code," *arXiv preprint arXiv:2107.03374*, 2021.
- [9] F. Calefato, F. Lanubile, and N. Novielli, "Ask me anything: A study of stack overflow questions related to software testing," in *Proceedings of the 40th International Conference on Software Engineering*. ACM, 2018, pp. 886--896.
- [10] H. Mi et al., "An empirical study of the characteristics of popular stack overflow questions," in *Proceedings of the 24th Asia-Pacific Software Engineering Conference*. IEEE, 2017, pp. 144--153.
- [11] J. Cohen, *Statistical Power Analysis for the Behavioral Sciences*. Routledge, 1988.
- [12] A. Agresti, *An Introduction to Categorical Data Analysis*. Wiley, 2018.